

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

Este trabajo práctico permite conocer, distinguir, desarrollar y utilizar la o las implementaciones o estructuras de datos del tipo de dato abstracto Lista.

Este trabajo práctico también está pensado para que cada estudiante desarrolle habilidades de búsqueda y selección de información técnica necesaria para resolver los ejercicios.

**Importante:**

Cualquiera de los lenguajes de programación que se utilizan para realizar las prácticas seguramente facilita una clase genérica que permite crear objetos del tipo List<T>, sin embargo, dicha clase no publica todos los métodos que se explican en teoría. En las clases teóricas de Estructura de Datos se desarrollan métodos particulares para que los estudiantes comprendan los diferentes mecanismos u operaciones que pueden realizarse con estructuras de datos del tipo List<T>; consecuentemente los estudiantes deberán codificar las implementaciones que se encuentran en las publicaciones de la asignatura.

**Ejercicios propuestos**

1) Dada la implementación de Lista Enlazada Simple estudiada en clases, agregarle métodos genéricos que permitan:

- Agregar un elemento en una posición ingresada por el usuario.
- Eliminar un elemento ubicado en una posición ingresada por el usuario.

De manera que, estos dos nuevos métodos deberán quedar incluidos en la clase SimpleLinkedList. Controlar los casos en donde la lista está vacía o el valor ingresado está fuera de los límites de la misma.

En el programa principal (main) se debe:

- Comprobar el correcto funcionamiento creando una lista que contenga objetos de la clase Producto la cual posee código, nombre, precio y stock).
- Cargar la lista con varios productos y, mediante un menú, mostrar al usuario las opciones para agregar o eliminar un producto en una posición específica, mostrando la lista antes y después de cada operación.

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

**Preguntas sobre el problema**

- a) ¿Por qué es necesario recorrer la lista nodo por nodo para insertar o eliminar en una posición, a diferencia de un arreglo donde se accede directamente por índice?
  - b) ¿Qué sucede si se solicita insertar en la posición 0 de una lista vacía? ¿Y si se solicita eliminar en una posición igual al tamaño de la lista (fuera de rango)?
  - c) Si la lista tiene los elementos A, B y C y se pide insertar "D" en la posición 1. Sin ejecutar el código: ¿Cómo queda la lista? ¿Qué nodo apunta a "D" y a quién apunta "D"?
  - d) Al insertar un nodo entre dos existentes, ¿por qué es crucial mantener temporalmente la referencia del nodo siguiente antes de cambiar el puntero next del nodo actual?
- 2) Usando una lista doblemente enlazada cargada con objetos de la clase Cancion que contiene los atributos titulo, artista, anioLanzamiento y duracionSegundos, realizar lo siguiente:
- a) Escribir un método que devuelva la canción más antigua de la lista.
  - b) Crear y mostrar una lista de canciones cuyo artista coincida con uno específico ingresado por el usuario.
  - c) Crear una nueva clase llamada EstadisticaArtista con los atributos nombreArtista y totalReproducciones (suma de duraciones). Recorrer la lista inicial, sumar las duraciones de las canciones de cada artista y, con esta información, crear los objetos EstadisticaArtista y guardarlos en una nueva lista doble ordenada de mayor a menor por el total de segundos. Mostrar la lista generada.

En el programa principal (main) se debe:

- a) Cargar al menos 10 canciones de distintos años pero con algunos artistas repetidos para poder probar el punto c.
- b) Ejecutar los métodos solicitados, pidiendo al usuario el artista a filtrar y mostrando los resultados por consola.

**Indicaciones:**

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

**Preguntas sobre el problema**

- a) ¿Qué ventaja ofrece una lista doblemente enlazada frente a una simple al momento de recorrerla para sumar duraciones y generar la lista de estadísticas?
- b) ¿Qué ocurre si el artista ingresado para el inciso (b) no existe en la lista? ¿Qué retorna el método?

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

- c) Para insertar de manera ordenada en la nueva lista doble, un compañero propone buscar siempre el final e insertar allí, y luego ordenar la lista entera. Otro propone usar inserción ordenada (buscar el lugar correcto y enlazar los punteros next y prev). ¿Qué ventajas y desventajas tiene cada alternativa?
  - d) Si se quisiera eliminar un nodo de la lista doblemente enlazada basándose en el título de la canción, ¿cuántos punteros (next y prev) se deben actualizar como máximo en el caso general (un nodo del medio)?
- 3) Dada la clase Suscripcion con los atributos idSuscripcion, usuario, plan y fechaInicio, crear dos listas simples con objetos de dicha clase. Escribir un programa que realice lo siguiente:
- a) Crear una nueva lista con aquellas suscripciones cuyo usuario se encuentre presente en ambas listas (intersección).
  - b) Contar la cantidad total de suscripciones que tiene un usuario (ingresado por teclado) sumando ambas listas.
  - c) Realizar la unión de ambas listas en una nueva lista simple, dejando los elementos ordenados por fechaInicio.

En el programa principal (main) se debe:

- a) Cargar datos de prueba en ambas listas (asegurándose de que haya al menos un usuario repetido en ambas).
- b) Solicitar al usuario un nombre de usuario para contar sus suscripciones.
- c) Mostrar la lista intersección y la lista unión ordenada por fecha.

**Indicaciones:**

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso y validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

**Preguntas sobre el problema**

- a) Para la intersección (punto a), ¿es necesario recorrer la segunda lista completamente por cada elemento de la primera? ¿Cómo se optimizaría la búsqueda si las listas estuvieran ordenadas por usuario?
- b) ¿Qué sucede si ambas listas están vacías? ¿Qué devuelve el método de unión?
- c) Si un usuario tiene 3 suscripciones en la Lista 1 y 2 en la Lista 2. ¿Cuál es el resultado esperado del punto (b) y del punto (a)?
- d) Al unir ambas listas, ¿se deben crear nuevos objetos Suscripcion en memoria, o se pueden reutilizar las referencias de los nodos de las listas originales?

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

- 4) Crear la clase Tarea con los atributos: idTarea, titulo, responsable, fechaLimite y completada (booleano). Además, crear la clase GestorTareas y cargar una serie de tareas en una lista usando SimpleLinkedList<Tarea>, luego implementar los siguientes métodos:
- agregarTarea(Tarea tarea): agrega una tarea al final de la lista..
  - completarTarea(int idTarea): marca una tarea como completada.
  - tareasPorResponsable(String responsable): devuelve una nueva lista con las tareas de un responsable específico.
  - tareasPendientes(): muestra solo las tareas que no han sido completadas.
  - eliminarTarea(int idTarea): elimina la tarea de la lista.
  - tareasProximas(String responsable): devuelve una lista con las tareas futuras (fechaLimite posterior a la fecha actual) de un responsable.

En el programa principal (main) se debe:

- Cargar al menos 5 tareas.
- Probar cada uno de los métodos a través de un menú de opciones, mostrando los resultados por consola.

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

Preguntas sobre el problema

- En el punto (e) eliminarTarea(int idTarea), al recorrer la lista simple para eliminar un nodo, ¿por qué es necesario mantener una referencia al nodo "anterior" y no solo al nodo "actual"?
  - ¿Qué pasa si el idTarea a eliminar no existe en la lista? ¿El programa lanza un error o cómo debería manejarlo?
  - Un compañero propone que para tareasPendientes() se modifique la lista original eliminando las completadas. Otro propone crear una nueva lista con las pendientes. ¿Cuál respeta mejor el principio de no destruir datos del cliente?
  - Un alumno hace nodo.getValor().setCompletada(true) mientras recorre la lista. ¿Es esto correcto en Java?
- 5) Crear una clase Aspirante con los atributos dni, nombre, apellido, notaEscrito, notaOral y notaPractico. Luego, crear una lista simple o doble que almacene objetos Aspirante. A continuación, desarrollar métodos para:
- Calcular y mostrar la nota promedio de cada aspirante de la lista (considerando las tres notas).

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

- b) Devolver el aspirante con la nota promedio más alta.
- c) Crear una nueva lista que contenga a los aspirantes que aprobaron el proceso con una nota promedio mayor o igual a 7.

En el programa principal (main) se debe:

- a) Cargar los datos de varios aspirantes.
- b) Mostrar el promedio individual de cada uno, anunciar al aspirante con el promedio más alto y mostrar la lista de aspirantes aprobados con nota promedio mayor o igual a 7.

**Indicaciones:**

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

**Preguntas sobre el problema**

- a) Al crear la nueva lista de aprobados (punto c), ¿Se crean nuevos objetos en memoria o se comparten las referencias con la lista original? ¿Qué consecuencias trae esto si luego se modifica una nota en la lista original?
  - b) ¿Qué sucede si todos los aspirantes tienen un promedio menor a 7? ¿Qué devuelve el punto (b) si la lista está vacía?
  - c) Si la lista tiene aspirantes con promedios 8, 6, 9 y 7, sin ejecutar: ¿Cuál es el promedio más alto? ¿Cuántos elementos tiene la lista de aprobados?
  - d) ¿El cálculo del promedio debería ser un método de la clase Aspirante o parte de la lógica interna del recorrido de la lista?
- 6) Crear una lista que almacene N números enteros generados de manera aleatoria y desarrollar métodos que realicen lo siguiente:
- a) Crear una nueva lista en donde los números negativos estén al principio y los positivos (incluyendo el cero) al final.
  - b) Solicitar al usuario el ingreso de dos valores (A y B) y devolver la suma de todos los elementos de la lista cuyo valor esté en el rango [A, B].
  - c) Crear una nueva lista que almacene los números de la lista inicial de manera ordenada (ascendente).

En el programa principal (main) se debe:

- a) Solicitar N, generar los aleatorios y mostrar la lista original.
- b) Ejecutar los 3 puntos del enunciado, mostrando claramente cada lista resultante y el resultado de la suma.

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

Preguntas sobre el problema

- Para el punto (a), si creas una nueva lista e insertas los negativos al inicio y los positivos al final, ¿se mantiene el orden original de cada grupo? Justifique.
- ¿Qué ocurre si en el punto (b) el usuario ingresa A=10 y B=5 (A mayor que B)? ¿Cómo debería reaccionar el programa?
- Para el punto (c), un compañero propone copiar la lista a un arreglo, ordenar el arreglo y volver a pasarlo a la lista. Otro propone usar inserción ordenada (recorrer la nueva lista hasta encontrar el lugar correcto y enlazar los nodos). ¿Cuál demuestra mejor dominio de las estructuras dinámicas?
- Si se quisiera eliminar los duplicados de la lista resultante del punto (c), ¿cómo se debería recorrer la lista ya ordenada para hacerlo eficientemente en una sola pasada?

- Codificar una implementación de la clase Queue<ELEMENT> (Cola genérica) utilizando en la estructura interna una lista genérica.

Usando dicha implementación, simular la cola de pedidos de un drive-thru (lugar donde se pide comida desde el auto). Se debe solicitar al usuario el ingreso de N números aleatorios entre 0 y 50 que representan el número de mesa/box que hizo el pedido.

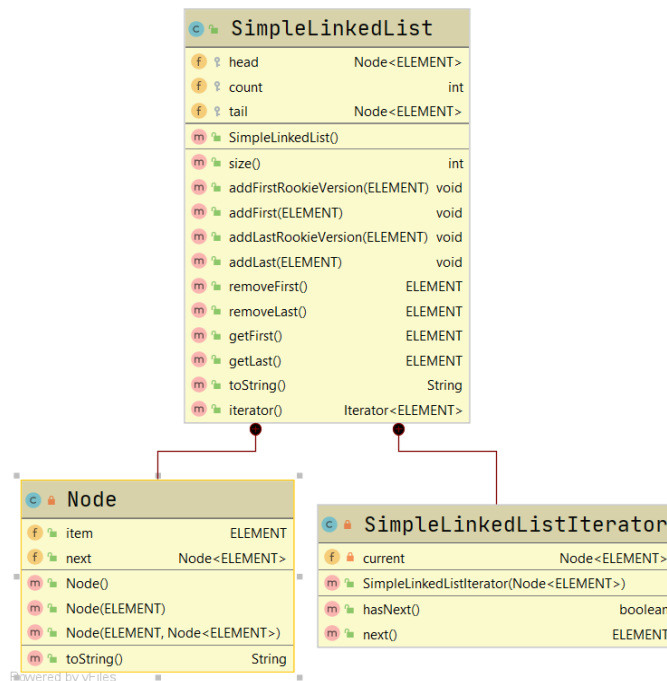
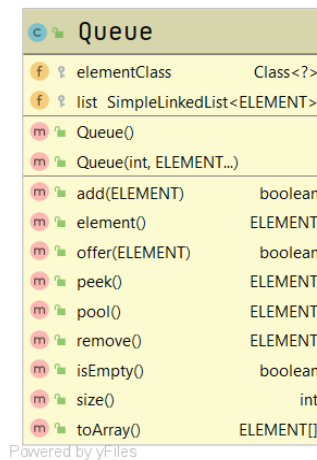
Si el valor ingresado es 0, significa que ese pedido fue "Cancelado" por el cliente. Se pide procesar la cola y eliminar todos los pedidos cancelados (todos los valores 0) de la estructura, dejando solo los pedidos válidos en el mismo orden en que llegaron. Al finalizar, se debe calcular el promedio de los números de mesa de los pedidos válidos que quedaron en la cola.

En el programa principal (main) se debe:

- Instanciar la Queue propia.
- Solicitar N al usuario y cargar los valores aleatorios.
- Procesar la cola aplicando la lógica de eliminación de cancelaciones.
- Mostrar la cola resultante (solo pedidos válidos) y el promedio de los números de mesa calculados.

Para la implementación de la clase, tener en cuenta los diagramas presentados. En el primer diagrama se muestra el campo interno "list" de la clase Queue<ELEMENT> y en el segundo se presenta los detalles de la clase SimpleLinkedList<ELEMENT>.

|                                                                                                                                                                    |                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Estructura de Datos                                                                                                                                                | <p align="center"><b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b></p> <p align="center"><b>Tema: Lista</b></p> |
| <p align="center"><b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b></p> <p align="center"><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b></p> |                                                                                                          |



#### Indicaciones:

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**. El ejercicio debe implementar un mecanismo para el ingreso de valores generados aleatoriamente.

**Preguntas sobre el problema**

- a) En el TP anterior se implementó una Cola Circular usando un arreglo fijo. Al cambiar la implementación para usar una SimpleLinkedList, ¿qué diferencia fundamental existe en la capacidad máxima de la cola y en el manejo de los índices head y tail? ¿Este tipo de cola puede estar llena?
  - b) Si la cola de pedidos está vacía y se llama al método para calcular el promedio, ¿cómo se evita una división por cero?
  - c) Si la cola tiene 5, 0, 8, 0 y 9, sin ejecutar: ¿Qué queda en la cola auxiliar después de procesarla para eliminar los ceros?
  - d) Al calcular el promedio, si el método desencola de la cola auxiliar y vuelve a encolar en la original. ¿Qué pasaría con el orden de los elementos si se encolara directamente en la original en lugar de usar una cola final?
- 8) Codificar una implementación de la clase Stack<T> (Pila genérica) utilizando en la estructura interna una lista genérica. Usando dicha implementación, cargar una pila con números enteros aleatorios y desarrollar un método que duplique los valores múltiplos de 3 que se encuentren en la pila. El orden relativo de los demás elementos no debe alterarse.

En el programa principal (main) se debe:

- a) Instanciar la Stack propia.
- b) Cargar valores (aleatorios o por teclado).
- c) Mostrar la pila original.
- d) Aplicar la lógica de duplicación y mostrar la pila resultante para verificar que el orden se mantuvo y los múltiplos de 3 se duplicaron.

**Indicaciones:**

Este ejercicio necesita del objeto scanner para ingresar datos por la consola o teclado, se espera que el código **controle los problemas** que normalmente ocurren al operar con la consola o teclado.

Se espera una **correcta modularización** entre el código que realiza el **ingreso** y **validación** de los datos respecto del código que hace lo que se **solicita en el ejercicio**.

El ejercicio debe implementar un mecanismo para seleccionar el ingreso de valores por consola o generados aleatoriamente.

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

**Preguntas sobre el problema**

- Si la implementación de Stack usa una SimpleLinkedList internamente, ¿es mejor insertar y eliminar en el inicio o en el final de la lista simple para respetar el orden LIFO de manera eficiente? Justifique.
- ¿Qué sucede si la pila está vacía al momento de intentar duplicar los múltiplos de 3?
- Si la pila original (cima -> base) es 3, 4 y 9, y se usa una pila auxiliar para recorrer y duplicar. Sin ejecutar: ¿Qué queda en la pila auxiliar antes de devolverla a la original?
- Un alumno intenta duplicar los múltiplos de 3 modificando la pila original directamente dentro de un bucle while(!pila.isEmpty()) sin usar una pila auxiliar. ¿Qué problema acarrea esta forma de solución?

**Ejercicio 9: Análisis de Soluciones (Comparación de código)**

Enunciado:

Desarrollar un método estático externo que reciba una SimpleLinkedList<Integer> y devuelva la suma de todos sus elementos. La lista original no debe modificarse (ni su contenido ni su orden) al finalizar el método. Solo se permite usar los métodos públicos de la clase SimpleLinkedList.

A continuación, se presentan dos soluciones propuestas por distintos estudiantes.

**Solución A:**

```
public static int sumarElementosA(SimpleLinkedList<Integer> lista) {
    int suma = 0;
    for (Integer num : lista) {
        suma += num;
    }
    return suma;
}
```

**Solución B:**

```
public static int sumarElementosB(SimpleLinkedList<Integer> lista) {
    int suma = 0;
    int tamanoOriginal = lista.size();
    for (int i = 0; i < tamanoOriginal; i++) {
        Integer num = lista.removeFirst();
        suma += num;
        lista.addLast(num);
    }
    return suma;
}
```

|                                                                                                                               |                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------|
| <b>Estructura de Datos</b>                                                                                                    | <b>TRABAJO PRÁCTICO N° 5 - Ciclo 2026</b><br><br><b>Tema: Lista</b> |
| <b>INGENIERÍA INFORMÁTICA – LICENCIATURA EN SISTEMAS</b><br><br><b>FACULTAD DE INGENIERÍA – UNIVERSIDAD NACIONAL DE JUJUY</b> |                                                                     |

### Preguntas sobre el problema

- ¿Cuál de las dos respeta mejor las restricciones del ejercicio? Tenga en cuenta que la lista "debe quedar sin modificaciones". ¿La Solución B modifica la estructura interna de la lista durante el proceso? Aunque el orden final sea el mismo, ¿considera que "rotar" la lista respetó la restricción?
- ¿Cuál solución es más fácil de explicar a un compañero? Argumente sobre la simplicidad del iterador (for-each) frente al uso de removeFirst y addLast.
- ¿Qué ventajas y desventajas tiene cada una? Mencione el tiempo de ejecución.

### Ejercicio 10: Depuración de Código (Análisis de errores internos)

Enunciado:

Se desea agregar un método a la clase SimpleLinkedList (asumiendo que la lista almacena enteros) que elimine todos los números pares de la estructura. El método debe manipular los punteros next de los nodos directamente.

Solución propuesta por un alumno, agregada dentro de la clase SimpleLinkedList (¡Contiene errores!):

```
public void eliminarPares() {
    if (this.count == 0) return;

    Node<ELEMENT> actual = this.head;

    while (actual != null) {
        if ((int) actual.item % 2 == 0) {
            // Si es par, saltea el siguiente nodo
            actual.next = actual.next.next;
            this.count--;
        }
        actual = actual.next;
    }
}
```

### Preguntas sobre el problema

- Analice el código línea por línea e identifique los errores conceptuales sobre el manejo de punteros y nodos.
- Argumente por qué constituyen errores. (Pista: Hay un error garrafal cuando el primer elemento (la head) es par, un error de excepción si el nodo par es el último de la lista, y un error de salto de nodos al avanzar actual después de haber hecho una eliminación).
- ¿La solución responde completamente a lo solicitado por la consigna? ¿Qué le pasa a la lista si ejecutamos este código con los elementos 2, 4 y 5?